

Examen Parcial 1 — 2025

Sistemas Operativos

Tomás Rodeghiero

UNRC — Universidad Nacional de Río Cuarto

2025

Resumen

Resolución completa del Parcial 1 del año 2025 de la cátedra de Sistemas Operativos (UNRC). El parcial consta de siete ejercicios que cubren: (i) IPC síncrono en un microkernel, contando traps y context switches y comparándolos con un kernel monolítico; (ii) preguntas sobre pipes y syscalls que crean procesos; (iii) implementación de `lock/unlock` sin instrucciones atómicas y análisis de fallas en monoprocesador y multiprocesador; (iv) verdadero/falso sobre *multilevel feedback queue* (MLFQ); (v) ejemplo de deadlock con `lock/unlock`; (vi) verdadero/falso sobre paginación; (vii) cálculos sobre una arquitectura de 32 bits con páginas de 4 MB (cantidad de páginas, interpretación de la dirección virtual, tabla de un proceso y detección de page faults). Cada ejercicio se desarrolla con la estructura **Enunciado** → **Análisis** → **Desarrollo** → **Respuesta**.

Índice

1. Ejercicio 1 — IPC síncrono en microkernel (1 pto)	1
2. Ejercicio 2 — Comandos y syscalls (1 pto)	3
3. Ejercicio 3 — <code>lock/unlock</code> sin atomicidad (2 ptos)	4
4. Ejercicio 4 — V/F sobre MLFQ (1.5 ptos)	6
5. Ejercicio 5 — Ejemplo de deadlock con <code>lock/unlock</code> (1.5 ptos)	7
6. Ejercicio 6 — V/F sobre paginación (1 pto)	8
7. Ejercicio 7 — Paginación en arquitectura de 32 bits (2 ptos)	9
Resumen del parcial	11

1. Ejercicio 1 — IPC síncrono en microkernel (1 pto)

Enunciado. En un SO con diseño microkernel, un proceso U realiza una llamada al sistema de la forma `send(process_manager, "getpid")`. El microkernel reenvía el mensaje al `process_manager`. El mecanismo de mensajes es **síncrono** y se basa en interrupciones por software, de modo que cada operación `send` o `receive` causa un trap al microkernel.

(a) Determinar la cantidad de traps y de cambios de contexto entre procesos que requiere esta llamada al sistema hasta que el proceso recibe la respuesta.

(b) Compararlo con un kernel monolítico.

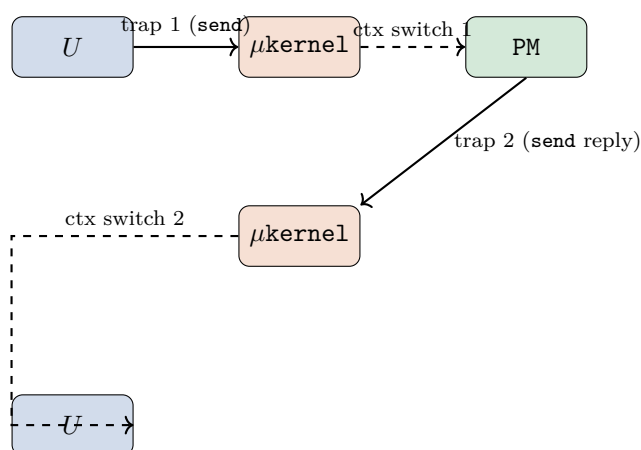
Análisis: arquitectura microkernel

En un microkernel los *servicios del SO* (gestor de memoria, file system, drivers, gestor de procesos, etc.) corren como procesos en **espacio de usuario**. La única función del microkernel mínimo es *enviar mensajes* entre procesos y gestionar la planificación y trampas de hardware.

Para que U obtenga su PID, no puede preguntar directamente al kernel (como en monolítico): tiene que *enviar un mensaje* al proceso servidor llamado **process_manager** (PM), que vive en modo usuario y responde.

1.a — Conteo de traps y context switches

Trazamos el camino completo de la llamada:



Secuencia mínima (send sincrónico, estilo Mach/L4/QNX, *send-and-receive-reply*):

1. U ejecuta **send**(PM, "getpid") en modo usuario. **Trap 1** al microkernel.
2. El microkernel encola el mensaje para PM. U queda bloqueado esperando la respuesta.
3. PM estaba previamente bloqueado en un **receive** (esperando peticiones). El microkernel lo despierta y ejecuta un **cambio de contexto 1** ($U \rightarrow PM$).
4. PM en modo usuario procesa la petición (computa el PID) y ejecuta **send**(U , "pid") para devolver la respuesta. **Trap 2** al microkernel.
5. El microkernel entrega la respuesta a U (que estaba bloqueado). **Cambio de contexto 2** ($PM \rightarrow U$).
6. U reanuda y obtiene la respuesta.

Respuesta

(a) La llamada requiere:

- **2 traps** al microkernel (uno por cada **send**).
- **2 cambios de contexto** entre procesos ($U \rightarrow PM$ y $PM \rightarrow U$).

Nota: si la API separara **send** y **receive** en syscalls independientes (estilo POSIX message queues), el conteo sería de 3 o 4 traps. La cantidad de cambios de contexto se mantiene en 2.

1.b — Comparación con kernel monolítico

En un kernel monolítico, **getpid()** es una syscall provista directamente por el kernel:

1. U ejecuta **getpid()**. **Trap 1** al kernel.
2. El kernel lee el PID del PCB del proceso actual y retorna.
3. U reanuda con el valor.

Total: 1 trap, 0 cambios de contexto.

Idea intuitiva

La comparación muestra el *overhead inherente* del modelo microkernel para syscalls simples: para obtener un valor que en monolítico se resuelve con ~ 50 ns de trap, el microkernel agrega $2\times$ cambio de contexto $+2\times$ trap, fácilmente $1\ \mu\text{s}$ o más. Por eso los microkernels (Mach, MINIX) son *conceptualmente elegantes* pero *históricamente más lentos* que los monolíticos para cargas típicas. L4 y derivados modernos (seL4, Fiasco) han optimizado mucho este camino, pero la diferencia estructural sigue existiendo.

2. Ejercicio 2 — Comandos y syscalls (1 pto)

2.a — “p1 | p2 hace que p2 se bloquee hasta que p1 escriba”

Respuesta

V. El operador `|` conecta el `stdout` de `p1` con el `stdin` de `p2` mediante un pipe del kernel. Si `p2` llama a `read(stdin, ...)` y el pipe está vacío, el `read` bloquea al proceso (estado `SLEEPING`) hasta que haya datos disponibles. Cuando `p1` escriba, el kernel despierta a `p2` y el `read` retorna.

2.b — “Si p1 se planifica primero, p1 se bloqueará al escribir”

Respuesta

F. `p1` no se bloquea al escribir mientras haya espacio en el buffer interno del pipe (`PIPE_BUF`, en Linux típicamente 64 KB). El escritor solo se bloquea cuando el buffer está *lleno* y nadie lo está consumiendo. Si `p1` escribe poco (o si `p2` lee a un ritmo razonable), no hay bloqueo del escritor: la idea del pipe es justamente desacoplar productor y consumidor.

Excepción: si `p1` es CPU-bound y produce gigas de datos sin que `p2` consuma, sí terminará bloqueándose.

2.c — Eliminar duplicados y ordenar sin temporales

Comando idiomático:

```
sort archivo | uniq > email-final.txt
```

O en una sola pasada con `sort -u`:

```
sort -u archivo > email-final.txt
```

Respuesta

La forma canónica es `sort archivo | uniq > email-final.txt`.

Por qué funciona sin archivos auxiliares:

- `sort` lee el archivo, lo ordena y vuelca a `stdout`.
- El pipe conecta esa salida con `uniq` como entrada.
- `uniq` requiere su entrada *ordenada* para funcionar (elimina duplicados *adyacentes*); por eso viene después de `sort`.
- `>` redirige la salida final al archivo `email-final.txt`.

Los datos viajan por buffers en RAM del kernel; no hay archivos intermedios en disco.

2.d — “fork y exec son las únicas que crean procesos”

Respuesta

F. `exec` no crea un proceso nuevo: reemplaza la imagen del proceso actual con la del nuevo programa (mismo PID, mismo descriptor). La única syscall que efectivamente *crea* un nuevo proceso es `fork()` (y sus variantes `vfork()`/`clone()` en Linux, internamente las tres descansan sobre el mismo mecanismo).

El patrón clásico **fork+exec** usa las dos juntas: `fork` crea el hijo, y `exec` lo transforma en otro programa. Pero la *creación* es solo del `fork`.

3. Ejercicio 3 — lock/unlock sin atomicidad (2 ptos)

Enunciado. Dar pseudo-código de `lock(&lk)` y `unlock(&lk)` que *no* deshabilitan ni habilitan interrupciones, asumiendo un sistema mono-procesador.

Pseudo-código (incorrecto a propósito)

```
typedef int lock_t; // 0 = libre, 1 = tomado

void lock(lock_t *lk) {
    while (*lk == 1) // 1) test
        ; // busy-wait
    *lk = 1; // 2) set
}

void unlock(lock_t *lk) {
    *lk = 0;
}
```

El problema fundamental: la pareja “test luego set” (líneas 1 y 2 de `lock`) **no es atómica**: entre la lectura de `*lk` y la escritura de `*lk = 1` puede ocurrir una interrupción que dé control a otro thread, el cual entre también a la sección crítica.

3.a — Traza que viola la exclusión mutua (monoprocesador)

Sean dos threads T_1 y T_2 en un único procesador. Inicialmente `lk = 0`.

t	Tarea ejecutando	Operación	Estado
t_1	T_1	<code>while (*lk == 1) → lee 0, sale</code>	$lk = 0$
t_2	interrupción del timer: scheduler cambia a T_2		
t_3	T_2	<code>while (*lk == 1) → lee 0, sale</code>	$lk = 0$
t_4	T_2	<code>*lk = 1</code>	$lk = 1$
t_5	T_2	ENTRA a la sección crítica	
t_6	interrupción del timer: scheduler cambia a T_1		
t_7	T_1	<code>*lk = 1 (siguiendo desde donde quedó)</code>	$lk = 1$
t_8	T_1	ENTRA a la sección crítica	

Resultado: ambos threads están simultáneamente dentro de la sección crítica. **No se garantiza exclusión mutua.**

La raíz del problema: T_1 *ya había decidido* entrar (`while` salió con $lk = 0$) pero *aún no había escrito* $lk = 1$. La interrupción dejó el “lock” en un estado inconsistente: T_1 comprometido pero aún no marcado.

3.b — Multiprocesador, aún con IRQs deshabilitadas

Ahora supongamos que `lock` *sí* deshabilita IRQs en la CPU local antes del test-set, y las habilita después. Pero el sistema es multiprocesador (varias CPUs corriendo en paralelo).

Problema: deshabilitar IRQs en la CPU A no impide que la CPU B ejecute simultáneamente. Las dos CPUs pueden estar en la sección de test-set al mismo tiempo, y como “test luego set” no es atómico a nivel de memoria, ambas terminan entrando.

Traza:

t	CPU A (corre T_1)	CPU B (corre T_2)	lk
t_1	<code>cli</code> (deshabilita IRQs en A)	<code>cli</code> (deshabilita IRQs en B)	0
t_2	lee <code>*lk</code> → 0, sale del <code>while</code>	lee <code>*lk</code> → 0, sale del <code>while</code>	0
t_3	<code>*lk = 1</code>	<code>*lk = 1</code> (lo mismo, ya estaba 1)	1
t_4	entra SC	entra SC	1

Ambas CPUs leen $lk = 0$ casi simultáneamente; ambas escriben `*lk = 1`; ambas entran a la sección crítica. **Falla.**

Importante

Conclusión: deshabilitar interrupciones funciona como mecanismo de exclusión *solo en monoprocesador*, porque allí la preemption es la única fuente de concurrencia. En multiprocesador, **además** hay paralelismo físico entre CPUs, y deshabilitar IRQs no afecta a otras CPUs.

La solución para multiprocesador es usar **instrucciones atómicas del hardware**:

- **Test-And-Set** (TAS): `xchg` en x86; actualiza el valor y retorna el anterior en una sola operación que el bus de memoria garantiza atómica.
- **Compare-And-Swap** (CAS): `cmpxchg`; condicional, base de muchos algoritmos lock-free.
- **LL/SC**: `ldrex/strex` (ARM), `lr/sc` (RISC-V); par de instrucciones cuya atomicidad la detecta el cache coherence.

Con cualquiera de estas, el “test luego set” se vuelve atómico y la exclusión mutua queda garantizada.

4. Ejercicio 4 — V/F sobre MLFQ (1.5 ptos)

Enunciado. Un SO usa MLFQ con la siguiente regla: si un proceso *no* usó todo su quantum, se le **sube** la prioridad (al nivel superior). Si lo usó completo, se le **baja** la prioridad. ¿V/F?

Análisis: qué procesos suben y cuáles bajan

- Un proceso **CPU-bound** ejecuta su quantum entero sin bloquearse (no hace I/O). Por la regla, **baja** de prioridad.
- Un proceso **I/O-bound** típicamente se bloquea en una operación de I/O (`read`, `recv`, etc.) antes de agotar el quantum. Por la regla, **sube** de prioridad.

Resultado: los procesos interactivos (I/O-bound) terminan en los niveles altos del MLFQ y se planifican *antes*; los CPU-bound quedan relegados a los niveles bajos.

4.a — “El scheduler prioriza a los procesos CPU-bound”

Respuesta

F. Es exactamente al revés. Los procesos CPU-bound consumen siempre todo su quantum y por eso *bajan* de prioridad; quedan en los niveles más bajos del MLFQ y se planifican menos.

4.b — “El scheduler prioriza a los procesos I/O-bound”

Respuesta

V. Los I/O-bound se bloquean antes de gastar su quantum, así que por la regla suben de prioridad. Tienden a quedar en los niveles altos, lo que les da menor tiempo de respuesta — ideal para procesos interactivos.

4.c — “El objetivo es maximizar el throughput”

Respuesta

F. El objetivo principal de MLFQ no es maximizar *throughput* (cantidad de procesos completados por unidad de tiempo). Para eso conviene FCFS o SJF (que minimizan el promedio de turnaround sin la sobrecarga de muchos cambios de contexto).

MLFQ se diseñó para optimizar el **tiempo de respuesta** de los procesos interactivos manteniendo *fairness*: la prioridad adaptativa hace que los procesos I/O-bound (que esperan input del usuario o de la red) se ejecuten apenas tienen trabajo, sin sacrificar totalmente a los CPU-bound.

4.d — “Minimizar el tiempo medio de espera de los I/O-bound”

Respuesta

V. Es exactamente el objetivo: que los procesos orientados a I/O (interactivos) reciban CPU rápidamente cuando están listos para ejecutar. Esto reduce su tiempo de espera en la cola `READY` y, por ende, mejora la latencia percibida por el usuario.

Un usuario tipeando en un editor o esperando una respuesta del servidor web nota fundamentalmente el tiempo entre que termina su input y la pantalla responde. MLFQ optimiza precisamente esa métrica.

5. Ejercicio 5 — Ejemplo de deadlock con lock/unlock (1.5 pts)

Enunciado. Dar un ejemplo de dos tareas (procesos o threads) que produzcan un deadlock usando las primitivas lock y unlock.

Análisis: el patrón clásico

La forma más simple de generar deadlock con dos mutex es que dos tareas los adquieran en **orden inverso**: una toma A y después espera B ; la otra toma B y después espera A .

Código

```
lock_t lock_A;
lock_t lock_B;

/* Tarea T1: toma A, luego B */
void *thread1(void *arg) {
    lock(&lock_A);
    /* aquí puede haber una interrupcion / context switch */
    lock(&lock_B); /* <-- espera B */

    /* seccion critica usando A y B */
    use(A, B);

    unlock(&lock_B);
    unlock(&lock_A);
    return NULL;
}

/* Tarea T2: toma B, luego A */
void *thread2(void *arg) {
    lock(&lock_B);
    /* aquí puede haber una interrupcion / context switch */
    lock(&lock_A); /* <-- espera A */

    /* seccion critica usando A y B */
    use(A, B);

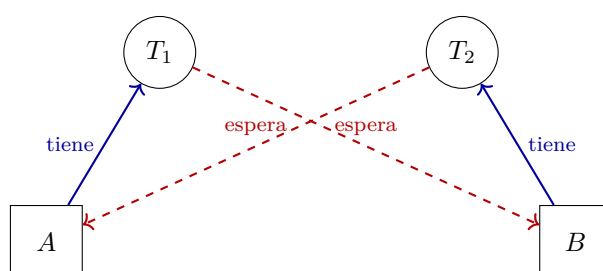
    unlock(&lock_A);
    unlock(&lock_B);
    return NULL;
}
```

Traza que produce el deadlock

t	T_1	T_2
t_1	lock(&lock_A) → obtiene A	—
t_2	(context switch)	
t_3	—	lock(&lock_B) → obtiene B
t_4	—	lock(&lock_A) → bloqueado
t_5	(context switch)	
t_6	lock(&lock_B) → bloqueado	

Resultado: T_1 tiene A y espera B . T_2 tiene B y espera A . Ninguno puede progresar; ambos están dormidos esperando un lock que el otro retiene. **Deadlock.**

Grafo de espera



Se cumple el ciclo $T_1 \rightarrow B \rightarrow T_2 \rightarrow A \rightarrow T_1$. Las cuatro condiciones de Coffman se satisfacen:

1. **Exclusión mutua:** cada lock es de uso exclusivo.
2. **Hold and wait:** cada tarea tiene un lock y espera otro.
3. **No preemption:** el SO no puede quitarle un lock a un thread.
4. **Espera circular:** el ciclo descrito.

Respuesta

El ejemplo presentado produce deadlock cuando T_1 adquiere `lock_A` y T_2 adquiere `lock_B` antes de que cualquiera intente tomar el segundo lock.

Prevención estándar: imponer un *orden global* de adquisición. Si ambas tareas tomaran siempre `lock_A` antes que `lock_B` (e.g., orden por ID o por dirección de memoria), el deadlock es imposible: la primera tarea en obtener A retiene la oportunidad de tomar también B sin que nadie esté pidiendo el otro en orden inverso.

6. Ejercicio 6 — V/F sobre paginación (1 pto)

6.a — “El paginado permite definir un espacio lógico a cada proceso”

Respuesta

V. Cada proceso tiene su propia tabla de páginas. La MMU, al hacer el cambio de contexto, carga el registro de tabla de páginas (`CR3` en x86, `satp` en RISC-V) con la dirección de la tabla del proceso entrante. A partir de ese momento, las direcciones virtuales que emite la CPU se traducen según esa tabla. Así, dos procesos pueden usar la misma dirección virtual `0x1000` sin colisionar: cada uno mapea a un frame físico distinto.

6.b — “Dos procesos que referencian la misma dirección virtual están compartiendo memoria”

Respuesta

F. Por defecto, dos procesos con sus tablas de páginas *independientes* mapean la *misma dirección virtual* a *frames físicos distintos*. Que dos procesos lean `0x1000` no significa nada en común: pueden estar referenciando memoria completamente distinta.

La memoria compartida real ocurre cuando dos tablas de páginas (de procesos distintos) tienen entradas que apuntan al *mismo frame físico*. Esto se logra explícitamente con `shmget/shmat`, `mmap(MAP_SHARED)`, librerías compartidas (`.so`), etc. Y nada obliga a que las direcciones virtuales coincidan entre procesos: típicamente *no* coinciden.

6.c — “Page fault cuando el bit V está apagado”

Respuesta

V. El bit V (valid) en la PTE indica si la entrada es legítima. Si $V = 0$, el hardware genera un *page fault* en cuanto la MMU intenta traducir esa dirección. Es el mecanismo básico para detectar:

- Accesos a direcciones no asignadas (typical NULL deref).
- Demand paging (la página existe lógicamente pero todavía no se cargó del disco).
- Copy-on-write (la página existe pero como solo-lectura marcada para CoW).
- Crecimiento dinámico del stack.

6.d — “Tabla de páginas implementa v2p: virt $\rightarrow$ phys”

Respuesta

V. La función conceptual de la tabla de páginas es exactamente esa: dado un **vpn** (los bits altos de la dirección virtual), devolver el **ppn** correspondiente y concatenar con el offset para formar la dirección física. Es una función determinista mientras el contenido de la tabla no cambie.

6.e — “El SO debería hacer panic ante un page fault de usuario”

Respuesta

F. Un page fault *no es un error fatal*. Es el mecanismo principal por el cual el SO implementa varias optimizaciones y servicios:

- **Demand paging:** solo se cargan en RAM las páginas efectivamente accedidas; las demás generan page fault que el kernel atiende cargándolas del disco.
- **Copy-on-write:** el fork() inicial no copia nada; las páginas se marcan solo-lectura. Al primer write, un page fault dispara la copia real.
- **Crecimiento dinámico del stack/heap:** cuando un proceso accede a una página recién asignada, el kernel la aloja en el momento.
- **Mapeos de archivos (mmap):** el contenido del archivo se trae a memoria solo cuando se referencia.

El SO debe **atender** el page fault. Si la dirección es *realmente inválida* (no pertenece a ningún VMA del proceso), recién entonces envía SIGSEGV al proceso, que normalmente termina. **Nunca debe hacer panic:** un panic significa falla del kernel, no de un proceso de usuario.

7. Ejercicio 7 — Paginación en arquitectura de 32 bits (2 ptos)

Enunciado. En una arquitectura de 32 bits, con páginas de 4 MB, los procesos usan un espacio de direcciones $[0, 0x8000000)$. Ayuda: $4\text{ MB} = 0x400000$.

Análisis de parámetros

- Tamaño de página: $4\text{ MB} = 2^{22}$ bytes.
- Espacio de direcciones del proceso: $[0, 0x8000000) = 2^{27}$ bytes = 128 MB.
- Espacio direccionable de la arquitectura: 2^{32} bytes = 4 GB (mucho mayor que lo que usa cada proceso).

7.a — Cantidad de páginas

Páginas físicas totales en la arquitectura:

$$2^{32} \div 2^{22} = 2^{10} = 1024 \text{ páginas físicas}$$

Páginas virtuales que usa el proceso:

$$0x8000000 \div 0x400000 = 2^{27} \div 2^{22} = 2^5 = 32 \text{ páginas}$$

Respuesta

(7.a) En total, la arquitectura permite hasta **1024 páginas** físicas (si toda la RAM estuviera presente). El proceso, restringido a $[0, 0x8000000)$, usa hasta **32 páginas virtuales**.

7.b — Interpretación de la dirección virtual

- Total: 32 bits.
- Offset (dentro de la página de 4 MB): 22 bits bajos.
- Page index (vpn): $32 - 22 = 10$ bits altos.



Cómo procesa la MMU una dirección virtual va :

1. Extrae los 10 bits altos $\rightarrow$ vpn.
2. Accede a la entrada **vpn** de la tabla de páginas.
3. Si $V = 0$ o se violan los permisos $\rightarrow$ page fault.
4. Si $V = 1$, concatena el **ppn** de la entrada con los 22 bits del offset $\rightarrow$ dirección física.

7.c — Tabla de páginas del proceso

El proceso usa: 2 páginas de código, 1 de datos globales, 1 libre (no mapeada), 2 de stack, en ese orden a partir de la dirección 0.

- **vpn 0**: código (página 1).
- **vpn 1**: código (página 2).
- **vpn 2**: datos globales.
- **vpn 3**: **libre (no mapeada)**.
- **vpn 4**: stack (página 1).
- **vpn 5**: stack (página 2).
- **vpn 6 en adelante**: no asignadas.

Asignamos los **ppn** arbitrariamente (los elige el SO):

vpn	ppn	V	R/W/X	Rango virtual	Área
0	0x10	1	R + X	[0x0000000, 0x0400000)	código (página 1)
1	0x11	1	R + X	[0x0400000, 0x0800000)	código (página 2)
2	0x20	1	R + W	[0x0800000, 0x0C00000)	datos globales
3	—	0	—	[0x0C00000, 0x1000000)	libre (no mapeada)
4	0x30	1	R + W	[0x1000000, 0x1400000)	stack (página 1)
5	0x31	1	R + W	[0x1400000, 0x1800000)	stack (página 2)
6-31	—	0	—	[0x1800000, 0x8000000)	no asignadas

7.d — ¿La dirección 0xc40000 produce page fault?

Descomposición (10 bits vpn, 22 bits offset):

$$0xc40000 = 0x00c40000 \text{ (con 32 bits)}$$

$$0xc40000 \div 0x400000 = 3 \text{ (con resto)}$$

$$0xc40000 - 3 \times 0x400000 = 0xc40000 - 0xc00000 = 0x40000$$

Por tanto: $\text{vpn} = 3$, $\text{offset} = 0x40000$.

Consulta a la tabla: la entrada $\text{vpn} = 3$ es la página “libre, no mapeada” con $V = 0$.

Respuesta

(7.d) Sí, produce page fault. La dirección 0xc40000 cae en la $\text{vpn} = 3$, que en la tabla del proceso tiene $V = 0$ (es la página intencionalmente libre entre datos y stack). El hardware levanta page fault y, como el proceso no espera tener esa dirección mapeada, el kernel le envía SIGSEGV.

7.e — ¿A qué área pertenece $\text{va} = 0x1400000$?

Descomposición:

$$0x1400000 \div 0x400000 = 5 \text{ (sin resto)}$$

$$\Rightarrow \text{vpn} = 5, \text{offset} = 0.$$

Consulta a la tabla: la entrada $\text{vpn} = 5$ corresponde a la segunda página del stack.

Respuesta

(7.e) 0x1400000 pertenece al área de **stack** (segunda página). Más concretamente, es el primer byte de la segunda página de stack: $\text{vpn} = 5$, $\text{offset} = 0$.

Nota: el enunciado dice “código, texto o stack”. Como “código” y “texto” son el mismo concepto (la sección `.text` del ejecutable), suponemos que la opción intermedia debía decir “datos”. La respuesta correcta es claramente **stack**.

Resumen del parcial

#	Tema	Respuesta sintética
1	Microkernel IPC	2 traps (send en ambos lados) + 2 ctx switches; monolítico: 1 trap, 0 ctx
2	Comandos y syscalls	a:V, b:F (buffer del pipe), c: <code>sort F uniq > final</code> , d:F (exec no crea)
3	locks no atómicos	Sin atomicidad: traza interrupciones entre test y set. En SMP con IRQs off: traza simultánea en dos CPUs
4	MLFQ V/F	a:F (no CPU-bound), b:V (sí I/O-bound), c:F (no es throughput), d:V (sí latencia interactiva)
5	Deadlock	T1 toma A,B; T2 toma B,A en orden inverso. 4 condiciones de Coffman. Fix: orden global
6	Paginación V/F	a:V, b:F (tablas distintas), c:V, d:V, e:F (no panic, atender el fault)
7	Paginación numérica	a: 1024 físicas / 32 virtuales; b: 10 + 22 bits; c: vpns 0,1,2,4,5 mapeadas y 3 libre; d: page fault; e: stack